

Weekly Authority Newsletter

Strategy, content model, operating process and build plan for an automated weekly email to our opted-in audience.

DOCUMENT TYPE	STATUS	PREPARED BY
Strategy & implementation plan	Draft for review	Growth Operations
PROGRAMME	AUDIENCE	REVISION
E-02 Weekly Authority Newsletter	Leadership & project team	1.0 — Draft

This document sets out what we intend to build, why, and the order in which it gets built. It is a plan for review and sign-off, not a status report. Items shown as decisions to lock require a named owner before the first live send.

Contents

FRONT MATTER

Executive summary

PART I — WHY WE ARE DOING THIS

1 The business case

2 Why manual does not scale, and why naive automation is dangerous

3 What success looks like

PART II — WHAT WE SEND

4 Audience and editorial promise

5 Content strategy: two streams

6 How an issue is selected

7 Anatomy of an issue

PART III — HOW IT RUNS

8 The operating rhythm

9 The approval gate

10 Controls and stop switches

11 Compliance and deliverability

12 Use of AI and editorial integrity

PART IV — HOW WE BUILD IT

13 Build sequence

14 Decisions to lock

15 Verification before the first live send

16 Risk register

17 Success metrics

18 Roles and responsibilities

APPENDICES

A System design

B Operating procedures

Executive summary

We intend to publish a weekly email to our opted-in audience that combines the articles we have published with the security issues that actually matter that week — assembled automatically, approved by a named person, and sent through our email platform.

The opportunity

A weekly, genuinely useful email is one of the few marketing channels we own outright. It does not depend on a social platform's ranking, it reaches people who have explicitly asked to hear from us, and its value compounds: showing up every week with something worth reading is what turns a vendor into a trusted advisor. We already produce the raw material. What we lack is a reliable way to package and ship it every single week without it becoming somebody's manual chore.

The problem worth naming

Automating email to a real list is not the same as automating an internal report. The failure modes are public and expensive: sending twice, sending before a human has read it, publishing a technical claim that is wrong, or breaking the unsubscribe mechanics that mailbox providers now require. Any of these damages the sender reputation of our domain, which is slow and costly to repair, and some carry regulatory exposure.

The plan therefore treats safety as the primary design problem rather than an afterthought. The system is built so that the dangerous outcomes are not merely unlikely but structurally prevented.

What we are proposing

- **A defined content model.** Each issue draws on two streams: our own published articles, and public vulnerability intelligence. Both are filtered against editorial criteria and capped, so an issue is consistent in size and shape.
- **An automated assembly line with a human gate.** The system gathers, selects, drafts and quality-checks the issue on its own. It cannot send. A named approver must actively approve each issue before anything reaches the list.
- **Compliance built in, not bolted on.** Postal address, honest sender identity, one-click unsubscribe and domain authentication are treated as launch prerequisites, and the system refuses to send if they are missing.
- **A staged build with a formal go-live gate.** Nothing reaches the real list until a defined set of checks has been passed and a person has authorised that first send.

THE CENTRAL DESIGN DECISION

The system is deliberately built so that it **cannot send without a human action**, and so that this cannot be quietly removed later. Everything else in this plan follows from that choice. If we ever find ourselves wanting to automate the approval away, the correct response is to question the reason, not the gate.

What we are asking for

Review and sign-off on the strategy in Parts I–III, and owners assigned to the decisions listed in section 14. Those decisions — our legal sending identity, the sending domain, the audience segment, the approver roster — are the genuine dependencies. They are business decisions rather than technical ones, and the project cannot reach a live send without them.

Why we are doing this

The commercial reasoning, the failure modes that make this harder than it looks, and the outcomes we intend to be judged against.

1 The business case

Our audience is made up of people responsible for keeping an organisation's technology working and secure. They are busy, they are marketed to constantly, and they are sceptical of vendors. They do not want a sales email. They will, however, reliably open something that tells them what broke this week and what to do about it.

Why email, and why weekly

Email is the only channel in our marketing mix that we genuinely own. A social platform can change its ranking and erase our reach without warning; a search engine can reshuffle results. A list of people who asked to hear from us is an asset that stays on our balance sheet. Weekly is the cadence that matches how the underlying subject matter moves — vulnerability disclosure is continuous, and a monthly digest is stale before it lands — and it is frequent enough to build the habit of recognising our name without becoming noise.

What the newsletter is for

AUTHORITY

Consistently useful commentary positions us as the people who understand this material, rather than the people who sell a product adjacent to it.

DISTRIBUTION FOR OUR OWN WORK

Articles we publish deserve more than one moment of attention. The newsletter gives every piece a second, larger audience.

STAYING PRESENT BETWEEN BUYING MOMENTS

Most of the list is not in market today. The point is to be the obvious call when they are.

A COMPOUNDING ASSET

List quality improves as disengaged addresses are retired and engaged readers stay. The channel gets better with age if it is looked after.

What it is explicitly not

Being clear about this protects the channel. The newsletter is **not** a lead-generation blast, **not** a product announcement vehicle, and **not** a destination for purchased or scraped contacts. It goes only to people who have explicitly opted in. Sending to a cold list from the same domain would destroy the deliverability of everything else we send, including the mail our sales and support teams depend on.

A DEPENDENCY WORTH STATING UP FRONT

The newsletter consumes articles that our content production process publishes. It does not write them. If article production stops, the newsletter has only the public intelligence stream to work with, and its distinctive value drops sharply. Content production is a genuine upstream dependency and should be resourced accordingly.

2 Why manual does not scale, and why naive automation is dangerous

The manual version fails quietly

Assembling a weekly issue by hand is perfectly possible once. It is possible for a few weeks. What happens next is predictable: the person responsible has a busy week, the issue slips, then it slips again, and the cadence that made the channel valuable is gone. Manual assembly also produces inconsistency — different structure, different tone, different length — which readers notice even when they cannot articulate it. The work is repetitive, unglamorous and easy to deprioritise, which is exactly the profile of work that should be automated.

The naive automation version fails loudly

The obvious shortcut is to wire a feed straight into an email platform and let it send on a schedule. Most email platforms offer this. It is the wrong choice here, and it is worth being precise about why, because each of these failure modes shaped a specific decision later in this plan.

FAILURE MODE	HOW IT HAPPENS	CONSEQUENCE
Duplicate send	Two scheduled jobs overlap, or a job is retried after a timeout that actually succeeded.	The same audience receives the issue twice. Complaint rate spikes; readers unsubscribe.
Sent without a human ever reading it	Approval is a link in an email. Corporate mail security scanners open every link in every message automatically to check it is safe. A link that sends when opened will be triggered by a machine, not a person.	An unreviewed issue goes to the entire list. This is the most damaging failure and the least obvious in advance.
Confidently wrong technical claims	An automated summariser invents a vulnerability identifier, misattributes a product, or links somewhere it was never given.	We publish false security information to security-literate readers. Credibility loss is immediate and hard to recover.
Broken unsubscribe mechanics	The message lacks the machine-readable one-click unsubscribe headers that major mailbox providers now require of bulk senders.	Filtering into spam, and regulatory exposure. Affects our domain, not just this campaign.
Sending to the wrong audience	An audience filter is misconfigured and resolves to everyone rather than the opted-in segment.	Commercial mail to people who never consented. The worst outcome on this list.

None of these are exotic. They are the ordinary failure modes of the ordinary approach. The design in Part III exists to make each one structurally impossible rather than merely unlikely.

3 What success looks like

Defining this before the mechanism keeps us honest about what we are buying. We should judge the project against outcomes, not against whether the automation is clever.

OUTCOME	WHAT IT MEANS IN PRACTICE
It ships every week	The cadence holds without heroics. Preparing an issue is a review task, not a production task.
Readers stay	Unsubscribe and complaint rates stay well inside mailbox-provider thresholds, and engaged readers keep engaging.
It lands in the inbox	Domain authentication is correct and sender reputation is healthy, so the mail arrives where people will see it.
Nothing embarrassing ever goes out	No duplicate sends, no unreviewed issues, no fabricated technical claims. Measured by absence of incidents.
It is not one person's private knowledge	The process is documented well enough that somebody else can run a week without a handover conversation.

THE HONEST BAR FOR THE FIRST PHASE

Success at launch is a small, clean, reliably delivered issue reaching a genuinely opted-in audience — not a large list or an impressive open rate. Reach is straightforward to grow later on a healthy sending reputation. Repairing a damaged one is not.

What we send

The reader, the promise we make to them, where the content comes from, and how a single issue is chosen and shaped.

4 Audience and editorial promise

Who is on the list

The list is made up of people who have explicitly opted in and who carry responsibility for an organisation's technology and security posture. In practice that is technical decision-makers and the people who advise them. Our editorial subject matter — ransomware, remote access and VPN exposure, identity and access, productivity-suite security, perimeter and firewall issues, and actively exploited vulnerabilities — reflects what this audience is accountable for.

Everyone receiving the newsletter has consented to receive it. There is no cold segment, no purchased list, and no automatic enrolment from unrelated activity. This is a hard constraint, not a preference.

The promise

WHAT A READER GETS, EVERY WEEK

A short, scannable briefing: what we published that is worth their time, and which security issues are being actively exploited right now — written plainly, linked to the source, and never padded to fill space. If a week is quiet, the issue is short. It never pretends otherwise.

Why they stay subscribed

- **It respects their time.** Capped length, consistent structure, and a subject line that says what is inside so they can decide in one second.
- **It is genuinely useful even if they never buy.** The vulnerability content has standalone operational value.
- **It does not sell at them.** Our own articles are presented as reading, not as pitches.
- **Leaving is easy.** A visible unsubscribe and a working one-click mechanism. Making it hard to leave produces spam complaints instead of unsubscribes, which is far worse for us.

5 Content strategy: two streams

Every issue is built from two clearly separated sources. Keeping them separate matters: they have different licensing constraints, different trust levels, and different editorial treatment.

STREAM ONE — OUR OWN PUBLISHED WORK

Articles we have published, consumed as links only. The newsletter never reaches into drafts and never publishes anything that is not already live on our own site.

Treated as: trusted in origin, still sanitised in handling.

STREAM TWO — PUBLIC VULNERABILITY INTELLIGENCE

The authoritative public catalogue of vulnerabilities known to be actively exploited, plus a small allowlist of additional feeds we choose deliberately.

Treated as: useful but untrusted content from an allowlisted source.

Editorial rules that apply to both

RULE	REASON
Original summaries, never full reprints	Republishing a third party's full article text is a copyright problem and adds no value. We write a short original summary and link to the source.
Public information only	No customer-specific detail and no internal threat indicators ever appear in a marketing email. This is a firm boundary between our marketing and our service delivery.
Attribution without implied endorsement	The public vulnerability catalogue is freely reusable, but the issuing agency's name and logo are not. We cite the source; we never imply they endorse us.
Every claim traceable to a source we ingested	If a vulnerability identifier or a link did not come from a source in this issue, it does not appear in the issue. This is enforced automatically, not by memory.
No link shorteners	Shortened links hide the destination, are treated as a spam signal, and break reader trust in exactly the audience least willing to click something opaque.

6 How an issue is selected

Selection is deliberately mechanical so that issues are consistent and the process is repeatable by anyone. Three filters apply in order.

Recency

Only material published inside a short rolling window is eligible. The window is slightly wider than the gap between issues, so that something published just after an issue is assembled is still caught by the next one rather than falling through the gap entirely.

Editorial relevance

Candidates are scored against our editorial keyword set — the subject areas our audience is accountable for. Items that do not touch our subject matter are not included merely because they are recent. The keyword set is a configuration decision, reviewed as our positioning evolves rather than fixed forever.

Caps

An issue carries at most **five** of our own articles and at most **seven** vulnerability items. The caps exist for three reasons: readers do not read past a certain length; a consistent size sets an expectation that makes the email easy to scan; and there is a hard technical ceiling to respect.

THE CLIPPING CEILING

Major mailbox providers truncate messages beyond roughly 102 kilobytes, hiding the remainder — including the footer that carries our postal address and unsubscribe link — behind a "view entire message" link. Since those elements are legal requirements, an issue that exceeds the ceiling is a compliance problem, not just a formatting one. The system measures each issue and refuses to proceed if it is over budget, with an earlier warning threshold so the trend is visible before it becomes a failure.

When there is not enough material

A quiet week is a normal outcome, not an error. The policy is to skip rather than to pad: an issue assembled from thin material teaches readers that our email is not worth opening. Skipping is recorded, so a skipped week is visibly a decision rather than a silent failure.

7 Anatomy of an issue

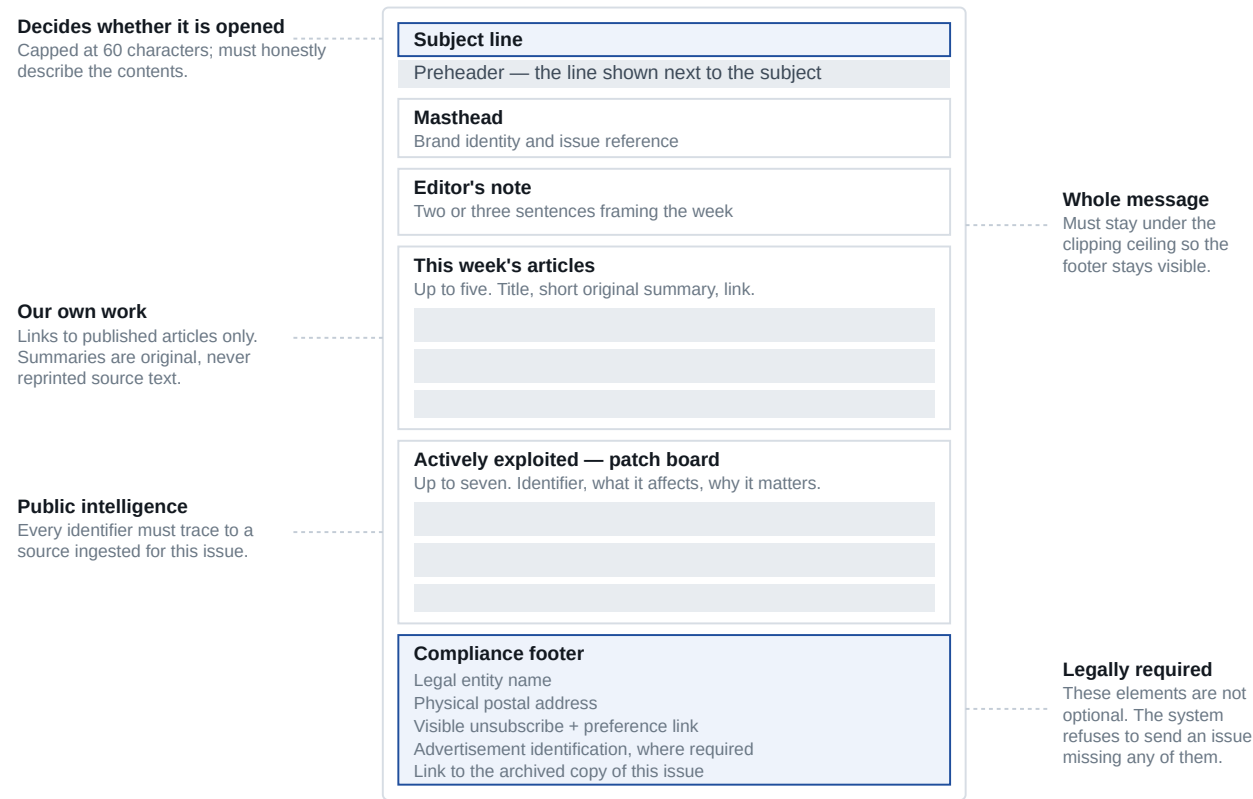


Figure 1 — Issue anatomy. Structure is fixed so readers learn where to look. The elements in the footer are legal obligations rather than design choices, which is why the assembly process treats a missing one as a blocking failure.

Subject and preheader discipline

The subject line is capped so it is not truncated on a phone, and it must describe what is actually in the issue. A subject that oversells is both a deliverability risk and a legal one: commercial email must not carry a misleading subject. The preheader is treated as part of the subject rather than as spare space, because together they are the entire basis on which a reader decides to open.

How it runs

The repeating process, the human gate at its centre, the controls available when something goes wrong, and the compliance position the whole thing has to satisfy.

8 The operating rhythm

Each issue moves through the same sequence. Everything up to the approval gate is automatic; everything past it is a consequence of a human decision.

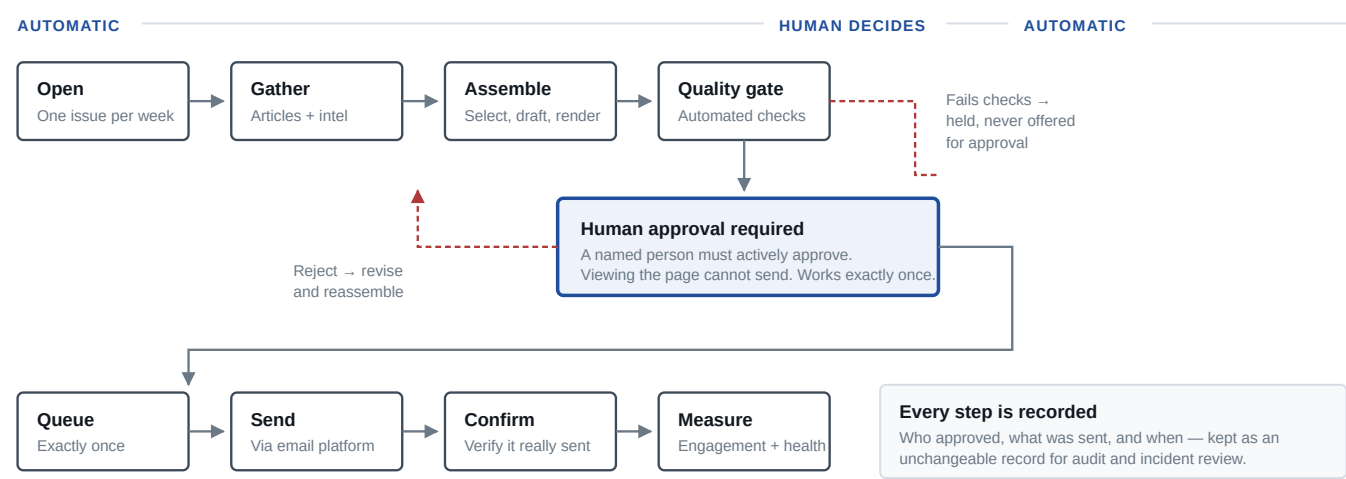


Figure 2 — The operating rhythm. The approval gate is the only point where the process waits for a person, and it is the only route to a send. Failures at the quality gate hold the issue back rather than passing a flawed issue to the approver.

What happens at each step

STEP	WHAT HAPPENS	OWNER	IF IT FAILS
Open	The week's issue is created. Exactly one exists per week; a repeated attempt joins the existing issue instead of starting a second one.	Automatic	Nothing is created twice. A duplicate attempt is a no-op.
Gather	Published articles and vulnerability intelligence are collected from approved sources only.	Automatic	Unreachable source is logged; the issue proceeds with what is available.
Assemble	Items are selected and capped, summaries drafted, and the issue rendered into its final form.	Automatic	Nothing to include → the week is skipped and recorded as skipped.

STEP	WHAT HAPPENS	OWNER	IF IT FAILS
Quality gate	Automated checks on links, technical identifiers, size, required footer elements and sender configuration.	Automatic	The issue is held and never offered for approval. The team is notified.
Approval	A named approver reviews the finished issue exactly as it will be received, and decides.	Named approver	Not approved within the agreed window → escalated. Never auto-approved.
Queue & send	The approved issue is queued once and handed to the email platform.	Automatic	Transient error → retried with a widening gap. A configuration error stops immediately and escalates rather than retrying pointlessly.
Confirm	The platform is asked what actually happened, and our record is updated to match reality.	Automatic	Mismatch is recorded for review rather than assumed away.
Measure	Delivery and engagement figures are collected against the health thresholds in section 11.	Automatic	Threshold breach triggers the response in section 10.

Skiping a week

Skiping is a supported, deliberate outcome — for a quiet week, a holiday period, or any week where we would rather send nothing than send something unreviewed. The standing rule is that skiping is always preferable to rushing an unreviewed issue out. The cost of a missed week is one missed week. The cost of a bad send is measured in list damage and sender reputation.

9 The approval gate

This section describes the single most important design decision in the project.

When an issue is ready, the approver receives a message containing a link. Opening that link shows the finished issue exactly as a subscriber would receive it, along with a button to approve. The critical property is that **opening the link does nothing at all**. Only pressing the button does anything.

WHY MERELY OPENING A LINK MUST NEVER SEND

Corporate mail security systems automatically open every link in every inbound message to check it is not malicious. They do this before a human sees the message. If our approval link performed the send when opened, the newsletter would go to the entire list the moment the approval message arrived in the approver's inbox — triggered by a scanner, with no human involvement whatsoever.

This is not a theoretical risk. It is the default behaviour of ordinary corporate email security, and it is the most common way that approval-by-link workflows fail in practice.

The properties the gate guarantees

PROPERTY	WHY IT MATTERS
Viewing changes nothing	Scanners, previews, forwarded copies and repeat visits are all harmless. The approver can open it as many times as they like.
Approval works exactly once	A second press — a double-click, a re-forwarded message, a stale browser tab — cannot produce a second send.
The link expires	An approval message that surfaces much later cannot be acted on. Stale approvals are not approvals.
What is approved is what is sent	The issue is frozen at approval. What the approver saw is byte-for-byte what goes out; it cannot be edited in between.
Approval is attributable	The record shows who approved, and when. Not "the system sent it".
The first live issues need two people	Two different named approvers must agree on the earliest production issues, when the risk of a configuration mistake is highest.

Reviewing, not rubber-stamping

The gate is only worth having if the review is real. The approver is asked to confirm that the subject honestly describes the contents, that the summaries are accurate and read as ours, that the technical claims are right, and that the footer is complete. Automated checks catch fabricated links and identifiers, but they cannot catch a summary that is plausible and wrong. That specific judgement is why a person is in the loop, and it is the part that must never be automated away.

10 Controls and stop switches

Three levels of stop, each blunter than the last. They exist so that the response to a problem is never "wait and hope".

LEVEL	STOPS	USE WHEN
Level 1	No new issues are started at all.	Something is wrong upstream — a bad content source, or a problem we have not diagnosed yet. Work in progress is untouched.
Level 2	Issues may still be prepared and approved, but nothing is handed to the email platform.	We are unsure about sending specifically. Preparation continues; delivery is held.
Level 3	Stops a campaign already handed to the email platform, from the platform's own controls.	Something is already in flight and must not go out.

STANDING RULE — CANCEL, NEVER RESUME

If a campaign is paused at the platform and its scheduled moment passes, resuming it does not reschedule it — it sends immediately, to everyone. Anyone touching a paused campaign must know this in advance. The rule is therefore absolute: **a paused campaign is cancelled and rebuilt, never resumed.** This is written into the operating procedures and rehearsed rather than left to be discovered during an incident.

Dry-run as the default posture

The system's default state is one where it performs every step of its work but does not actually send. Live sending is something that has to be deliberately switched on, with the audience, the environment and the stop switches all explicitly aligned. The consequence is that a mistake in configuration, a misfired scheduled job, or an accidental deployment produces a rehearsal rather than a send.

11 Compliance and deliverability

Commercial email is regulated, and the major mailbox providers now enforce their own requirements on top of the law. We follow the stricter of the two wherever they overlap.

Legal obligations

OBLIGATION	HOW WE MEET IT
Honest sender identity	The from-name, from-address and reply-to identify our real business. No disguised or misleading routing.
Subject matches the message	Editorial discipline plus an automated length cap. A subject that oversells is a legal problem, not just a marketing one.
Valid physical postal address	Present in the footer of every issue. This is a launch prerequisite: the system refuses to send while the address is unset.
Clear opt-out, promptly honoured	A visible unsubscribe in every issue, honoured within 48 hours — deliberately stricter than the legal minimum, because that is what mailbox providers expect of bulk senders.
Advertisement identification	Included where the primary purpose of the message is commercial.

RESPONSIBILITY CANNOT BE OUTSOURCED

Using a third-party email platform does not transfer legal responsibility for what we send. The obligations remain ours, which is why they are enforced in our own process rather than assumed to be handled downstream.

Mailbox provider requirements

Major providers apply requirements to bulk senders that determine whether mail reaches the inbox at all. We intend to meet these from the first issue, regardless of our volume, because building the practice at low volume is far easier than retrofitting it later.

- **Domain authentication.** The sending domain must be properly authenticated so receiving systems can verify the mail genuinely came from us and was not altered in transit.
- **A published policy for unauthenticated mail,** with the visible sender aligned to the authenticated domain.
- **One-click unsubscribe.** Beyond the visible link, the message must carry the machine-readable headers that let a mailbox provider offer its own unsubscribe control. This is a requirement, not an enhancement.
- **Complaint rate below the enforced ceiling.** Providers publish a hard threshold; we set our internal warning level well below it so we can act on a trend rather than react to a breach.

COMPLAINT THRESHOLDS AND THE RESPONSE

LEVEL	THRESHOLD	RESPONSE
WARNING	0.1% complaints	Investigate. Review recent acquisition sources and content. Do not treat as noise.
HARD STOP	0.3% complaints	Stop sending. No further issues until the cause is found and sender reputation recovers.

List hygiene

- **Opt-in only.** The audience is an explicit opt-in segment. There is no path by which a contact acquired for another purpose joins the newsletter automatically.
- **Suppression respected.** Unsubscribed, complained and hard-bounced addresses are suppressed at the platform and never re-added.
- **Sunset policy.** Subscribers with no engagement over an extended period — our working figure is 90 days — are re-permissioned or retired. Continuing to mail people who never open is what drags a sender reputation down, and a smaller engaged list outperforms a larger stale one.

Subscriber data

THE LIST STAYS WHERE IT LIVES

Subscriber records remain in the email platform. Our own systems store only what they need to operate: which issue was sent, which campaign it became, and aggregate counts. We deliberately avoid copying personal data into additional systems, because every copy is another thing to secure, to keep in sync, and to account for.

Content licensing

The public vulnerability catalogue we rely on is released for free reuse, which is why we can build on it. The issuing agency's name and insignia are a separate matter: we cite the source as a factual reference and never use official logos or any wording that could imply endorsement of our business. We also do not reproduce third-party article text in full — we summarise in our own words and link.

12 Use of AI and editorial integrity

Language models draft the summary copy. Being precise about what that does and does not mean matters, both for the quality of the product and because our readers are exactly the audience most likely to notice — and least likely to forgive — a confidently wrong technical claim.

WHAT THE MODEL DOES

Writes short summaries of items already selected, drafts a subject line and preheader, and writes a brief framing note for the issue.

WHAT THE MODEL NEVER DOES

Choose the audience. Decide whether to send. Fetch anything from the internet. Introduce a link or a vulnerability identifier that was not in the source material it was given.

Three defences against fabrication

1. **A closed set of source material.** The model is given the specific items selected for this issue and nothing else. It has no browsing ability and no wider corpus to draw a plausible-sounding detail from.
2. **Automated verification of every reference.** Every link and every vulnerability identifier in the finished issue is checked against the source material actually ingested. Anything that does not trace back is rejected and the issue is held. A fabricated reference cannot reach the approver, let alone a reader.
3. **Human review.** The approver reads the finished issue before anyone else does.

Treating source text as content, never as instructions

WHY THIS DESERVES ITS OWN PARAGRAPH

We ingest text from public feeds. Anyone who can publish to a feed can write whatever they like in it — including text crafted to read as an instruction to an automated system. Our design treats everything arriving from a feed strictly as material to summarise, never as direction to follow. Instructions to the model come only from a reviewed internal prompt that feed content cannot alter.

The residual risk we accept, stated plainly

Automated checks catch invented links and invented identifiers. They cannot catch a summary that is fluent, correctly cited, and subtly wrong about what a vulnerability actually does. No amount of additional automation resolves this, because the error is one of meaning rather than of form.

This is precisely why a knowledgeable human approves each issue, and it is the strongest argument against ever removing that step for convenience. We should be honest that this risk is managed rather than eliminated.

Model policy

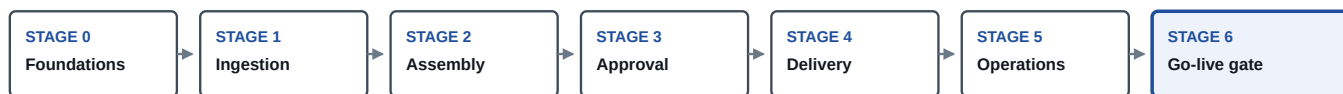
The model is a configuration choice rather than an architectural one, so it can be changed if quality or cost warrants it without redesigning anything. If the model is unavailable or returns something that fails validation, the issue is held for attention rather than sent with degraded copy.

How we build it

The order of work, the decisions that have to be made by people rather than by engineering, what must be proven before the first live send, and how we will know it is working.

13 Build sequence

The work is organised into stages ordered by dependency. Each stage produces something demonstrable, and each has a clear definition of done so that progress is visible rather than asserted. Stages are sequenced by what they depend on, not allocated to a calendar.



Stages 1 and 2 can proceed while the decisions in Stage 0 are still being settled; Stage 4 cannot begin until the sending platform decisions are made. Stage 6 depends on every preceding stage being complete.

Figure 3 — Build stages, ordered by dependency. The go-live gate is a distinct stage rather than a formality at the end of delivery work.

Stage 0 — Foundations

Objective: settle the decisions everything else depends on, and stand up the environment the system runs in.

Produces: a confirmed brand and sending identity, an authenticated sending domain, a named audience segment, a named approver roster, and a running environment with its data store.

Done when: the decisions in section 14 have named owners and confirmed values, and domain authentication is verified as correct.

Stage 1 — Content ingestion

Objective: reliably collect our published articles and public vulnerability intelligence from approved sources.

Produces: a normalised store of candidate items, with source restrictions in place so the system can only fetch from sources we have approved.

Done when: both streams ingest correctly, hostile or malformed input is handled safely rather than propagated, and the same item ingested twice does not appear twice.

Depends on: confirmed content sources from Stage 0.

Stage 2 — Assembly, rendering and quality gates

Objective: turn a set of candidate items into a finished, checked issue.

Produces: selection and capping, drafted summaries, the rendered email in both rich and plain-text form, and the automated quality checks.

Done when: a complete issue can be produced from sample material and the quality gates demonstrably reject a bad issue — a fabricated link, an oversized message, a missing footer element — rather than merely passing a good one.

Depends on: Stage 1.

Stage 3 — Approval and control

Objective: put the human gate in place, along with the stop switches.

Produces: the approval notification and review page, single-use approval, the record of who approved what, and the stop switches from section 10.

Done when: viewing the approval page provably changes nothing, approving works exactly once, and each stop switch has been exercised.

Depends on: Stage 2.

Stage 4 — Delivery integration and sandbox verification

Objective: connect to the email platform and establish, by observation rather than assumption, exactly how it behaves.

Produces: the integration, and a recorded body of evidence covering how the platform accepts an issue, how an audience is addressed, what it reports back, and what the message actually looks like when received.

Done when: a test issue has been sent to a small set of seed recipients we control, and each open question about platform behaviour has been answered from a real request and response rather than from documentation.

Depends on: Stage 3, and on sending-platform access from Stage 0.

WHY THIS STAGE IS A GENUINE UNKNOWN

Published documentation for the sending platform does not fully specify some of the details we depend on, and available sources disagree in places. These questions can only be settled by observing the platform's actual behaviour in a controlled test account. Until that is done, the correct engineering posture is to treat them as unresolved and refuse to send to the real audience — which is exactly what the system does.

Stage 5 — Operations

Objective: make the system supportable by someone other than the person who built it.

Produces: monitoring and alerting, escalation for anything stuck, backups with a tested restore, and written operating procedures.

Done when: a restore has actually been performed from a backup, and the incident procedures have been walked through as a rehearsal rather than only written down.

Depends on: Stage 3.

Stage 6 — Go-live gate

Objective: a deliberate, evidenced decision to send to the real audience for the first time.

Produces: a completed verification checklist (section 15) and a recorded authorisation naming the person who approved the first live send.

Done when: every check is passed and the authorisation is recorded. This gate is a human decision by design and is not automated.

Depends on: all preceding stages.

14 Decisions to lock

These are the genuine dependencies. They are business decisions rather than engineering tasks, and the project cannot reach a live send without them. Each needs a named owner.

DECISION	WHAT IS NEEDED	OWNER	BLOCKS
Legal sending entity	The registered legal or trading name that appears in the footer of every issue.	Leadership	Any send
Physical postal address	A valid physical address for the footer. A legal requirement for commercial email.	Leadership	Any send
Sending identity	From-name, from-address and reply-to. Decide whether replies reach a monitored mailbox — if we invite replies, someone must read them.	Growth Ops	Any send
Sending domain	Which domain or subdomain sends the newsletter, and confirmation that it is properly authenticated. Consider separating marketing mail from business-critical mail so a reputation problem in one does not affect the other.	Growth Ops + IT	Any send
Audience segment	Precisely which opt-in segment receives the newsletter, and how someone joins it. Must be demonstrably opt-in.	Growth Ops	Any send
Approver roster	Who may approve an issue, and who covers when they are unavailable. At least two named people, since the first live issues require two approvals.	Leadership	Stage 3
Schedule	Which day and hour issues arrive, in which timezone, and how far ahead of that an issue is prepared so there is a real review window.	Growth Ops	Stage 0
Content sources	The feed for our published articles, and confirmation of which additional intelligence feeds we want beyond the primary catalogue.	Growth Ops	Stage 1
Editorial keywords	The subject areas that make an item relevant to this audience. Sets what an issue contains.	Growth Ops	Stage 2

DECISION	WHAT IS NEEDED	OWNER	BLOCKS
Brand assets	Logo, colours and where assets are hosted, so the issue looks like us.	Marketing	Stage 2
Platform accounts	A test account for verification with seed recipients we control, and the live account. These must be separate.	Growth Ops	Stage 4
Escalation contact	Who is contacted when something is stuck or has failed, and by what route.	Leadership	Stage 5
Preference handling	Whether subscribers can choose which of our emails they receive, or only unsubscribe from everything.	Growth Ops	Stage 4

THE SHAPE OF THIS LIST IS DELIBERATE

Almost every blocking item is a business decision, not an engineering one. The system is built to refuse to operate on placeholder values rather than to quietly substitute something plausible — so an unmade decision surfaces as a clear blocked state rather than as an incorrect email arriving in somebody's inbox.

15 Verification before the first live send

Each item below must be *demonstrated*, not assumed. Several cannot be proven from documentation and require observing real behaviour.

CHECK	HOW IT IS PROVEN	WHY IT CANNOT BE ASSUMED
Platform behaviour	A test issue created and sent in a separate test account, with the actual requests and responses recorded.	Published documentation is incomplete and sources disagree.
Audience resolves narrowly	Confirm the audience selection resolves to exactly the seed recipients — before it is ever pointed at the real list.	An audience filter that silently resolves wider than intended is the worst failure available to us.
One-click unsubscribe present	Inspect the raw source of an actually received test message and confirm the required headers are there.	No documentation can prove what the platform put on a specific message. Only the received message can.
Domain authentication	Verify against a received message that authentication passes and the visible sender aligns.	A configuration that looks right can still fail in transit.
Inbox placement	Score the issue with a deliverability testing service and address anything flagged.	Catches problems before real subscribers experience them.
Rendering	View the issue in the major mail clients our audience actually uses, and confirm it is not truncated.	Email clients render inconsistently; a layout that works in one can break in another.
Approval gate	Open a real approval link from a real mailbox — including one behind corporate mail security — and confirm the issue's state is unchanged afterwards.	This is the property most worth proving under realistic conditions rather than in a test.

CHECK	HOW IT IS PROVEN	WHY IT CANNOT BE ASSUMED
Full rehearsal	Run a complete issue end to end with sending disabled.	Exercises the real process, including the people, with no possibility of a send.
Restore from backup	Actually restore a backup and confirm the result is complete.	An untested backup is not a backup.
Stop switches	Exercise each level and confirm it stops what it claims to.	An untested control is not a control.
Compliance footer	Confirm real legal name, real postal address and working unsubscribe in a received message.	These are legal requirements and the most consequential thing to get wrong.

THE FIRST LIVE SEND

Once every check above is passed, the first live send is a deliberate act by a named person, with a second approver agreeing. It is recorded. The system will not perform it on its own, and this is by design rather than by omission.

16 Risk register

RISK	IMPACT	MITIGATION	OWNER
Send to the wrong or wider audience	SEVERE	Audience proven to resolve to seed recipients before any live use; live sending requires environment, audience and stop switches all deliberately aligned; system refuses an empty or unverified audience definition.	Growth Ops
Issue sent without human review	SEVERE	Viewing the approval link cannot send; only an explicit action can. Proven under real corporate mail security before launch.	Growth Ops
Duplicate send	SEVERE	One issue per week enforced at the data layer; approval works exactly once; delivery is queued with a guard against repeat submission.	AI Eng
Factually wrong technical claim	SEVERE	Closed source material; automated verification of every link and identifier; knowledgeable human review. Meaning-level error remains a managed residual risk.	Approver
Sender reputation damage	SEVERE	Opt-in only; authentication before launch; complaint monitoring with a warning level below the enforced ceiling; sunset policy; consider separating marketing from business-critical mail.	Growth Ops + IT
Missing legal footer elements	SEVERE	Treated as launch prerequisites; the system refuses to send while any required brand or legal value is unset, checked again immediately before delivery.	Leadership

RISK	IMPACT	MITIGATION	OWNER
Platform behaves differently than expected	MODERATE	Resolved by observation in a test account before launch; unresolved questions block the live audience rather than being assumed away.	AI Eng
Article production stalls	MODERATE	Issues can run on the intelligence stream alone, but distinctiveness drops. Escalate rather than quietly degrade; skip rather than pad.	Marketing
Cadence slips	MODERATE	Preparation is automatic, so the only manual step is review; escalation when an issue sits unapproved past the agreed window.	Growth Ops
Hostile content in a public feed	MODERATE	Sources restricted to an approved list; incoming text sanitised; feed text never treated as instructions to the system.	AI Eng
Key-person dependency	MODERATE	Written operating procedures; more than one named approver; rehearsed incident handling. Worth addressing early rather than after it becomes a problem.	Leadership
Data loss	CONTAINED	Regular backups with a tested restore. Sent issues are immutable, so a restored copy is identical to what recipients received.	AI Eng

17 Success metrics

Measured from the first live issue. The distinction between what we *watch* and what we *act on* matters: acting on noise is its own failure mode.

AREA	MEASURE	WHAT WE DO ABOUT IT
Delivery health ACT ON	Complaint rate	Investigate at the warning level; stop sending at the enforced ceiling.
	Hard bounce rate	A rising trend indicates a list acquisition problem. Investigate the source.
	Delivery rate	A sudden drop suggests a reputation or authentication problem. Treat as urgent.
Engagement WATCH	Click rate, and clicks as a share of opens	The most honest available signal of whether content is landing. Guides editorial direction.
	Open rate	Directional only — see the caveat below. Never used alone to judge an issue.
List health WATCH	Net growth, unsubscribe rate	A spike after a specific issue is editorial feedback worth taking seriously.
	Share of list engaged	Drives the sunset policy. A smaller engaged list beats a larger stale one.

AREA	MEASURE	WHAT WE DO ABOUT IT
Operational ACT ON	Issues shipped as planned	The core reliability measure. A miss should have a recorded reason.
	Incidents	Target is zero. Any duplicate send, unreviewed send or factual correction is reviewed.

A CAVEAT ABOUT OPEN RATES

Open tracking has become substantially unreliable. Privacy features in major mail clients pre-fetch tracking pixels automatically, registering opens that never happened. Reported open rates are therefore inflated by an unknown amount and are not comparable across audiences or over time. We track the figure for direction only, and judge content on clicks, replies and unsubscribes. Anyone reporting on this channel should understand that before quoting an open rate.

18 Roles and responsibilities

ROLE	RESPONSIBLE FOR	NOTES
Leadership	Approving this plan. Confirming the legal entity, postal address and approver roster. Authorising the first live send.	The blocking decisions in section 14 sit here.
Growth Operations	Editorial direction and keyword set. Audience definition and list hygiene. Approving issues. Monitoring the health metrics.	Day-to-day owner of the channel.
Named approver(s)	Reviewing each issue before it sends. Judging accuracy and tone, not just presence.	At least two named people. Review must be genuine, not a rubber stamp.
AI Engineering	Building and maintaining the system. Content sources, quality gates, platform integration, backups, monitoring.	Does not decide what is sent or to whom.
IT	Domain authentication and DNS for the sending domain.	Needed at Stage 0 and again at verification.
Marketing	Brand assets. Article production, which the newsletter depends on.	Upstream dependency, not part of this project.
Escalation contact	Responding when an issue is stuck, has failed, or a health threshold is breached.	Must be a named person, not a shared inbox nobody watches.

ONE SEPARATION WORTH PRESERVING

The people who build the system do not decide what goes out or who receives it, and the people who approve content do not need to understand the system to do their job well. Keeping those responsibilities apart is what makes the approval gate meaningful rather than ceremonial.

Reference material

Supporting detail for the project team. Not required reading for the strategic decision in Parts I–III.

A System design

The system is organised into three separated concerns. The separation is the point: it is what lets us make the safety guarantees in Part III, and it is the main thing distinguishing this from wiring a feed directly into an email platform.

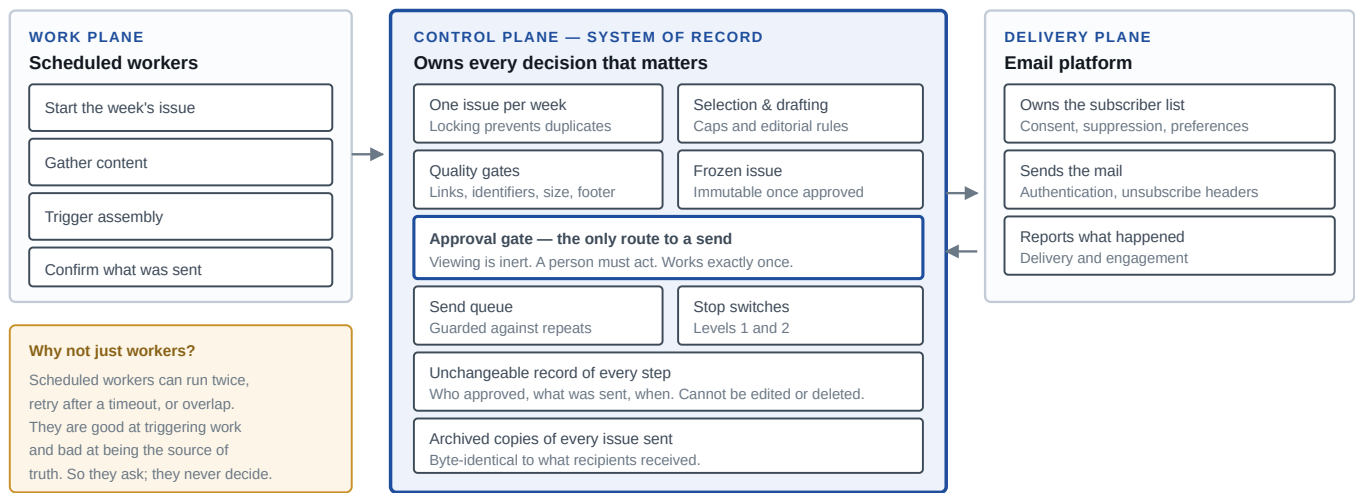


Figure 4 — Three planes. The control plane is the system of record: it owns the lifecycle, the approval gate and the audit trail. Workers only ask it to do things, and the email platform only receives an already-approved issue.

Why the separation matters

PLANE	RESPONSIBLE FOR	EXPLICITLY NOT RESPONSIBLE FOR
Work plane	Triggering activity on a schedule.	Deciding anything. It holds no state and is not trusted to be correct about what has already happened.
Control plane	The issue lifecycle, one-issue-per-week enforcement, quality gates, the approval gate, the audit trail, and the archive.	Holding subscriber personal data.
Delivery plane	The subscriber list, consent and suppression, and actually transmitting the mail.	Deciding what gets sent or when. It receives an already-approved issue.

Technology choices

The body of this document deliberately avoids naming products, because the strategy holds regardless of which vendor fills each role. For the project team, this is what fills them and why.

ROLE	CHOICE	WHY
System of record	A relational database (PostgreSQL)	We need guaranteed uniqueness, locking that prevents two jobs acting on the same issue, and an append-only history. A workflow tool cannot provide these.
Control plane	A small service we own	Holds the lifecycle, the approval gate and the audit trail. Deliberately ours rather than a vendor's, because these are the guarantees the whole design rests on.
Scheduled workers	A workflow automation tool (n8n)	Good at running things on a schedule and easy to change without a deploy. Trusted only to trigger work, never to decide anything or hold state.
Email platform	HighLevel LC Email	Already holds our contacts and consent records, and provides the one-click unsubscribe headers bulk senders are required to include. Keeping the list in one place avoids a second copy of subscriber data.
Summarisation	A hosted language model	Drafts summary copy. A configuration choice, changeable without redesign if quality or cost warrants.
CRM view	Twenty	Receives a read-only projection of what was sent. It is not the source of truth and has no ability to send.

ONE CONSTRAINT THE EMAIL PLATFORM IMPOSES

Sending must go through the platform's own email service rather than a custom mail server, because that is what supplies the required unsubscribe headers and the domain authentication that goes with them. The trade-off is that some details of the platform's behaviour are not fully documented, which is why Stage 4 verifies them by observation rather than trusting documentation.

Two properties worth understanding

Issues are frozen. At approval, the issue is fixed and identified by a fingerprint of its exact contents. What the approver saw is what is sent, and the archived copy is provably the same as what recipients received. There is no window in which an approved issue can be edited.

The record cannot be rewritten. The history of what happened to each issue is append-only: entries can be added but not altered or deleted, including by the system itself. If we ever need to answer "who approved this and what exactly went out", the answer is reliable.

B Operating procedures

A normal week

1. The issue is prepared automatically, ahead of the send, allowing a real review window.
2. The approver receives a notification with a link to the finished issue.

3. The approver reviews and either approves or rejects with a comment.
4. On approval, the issue is queued and handed to the email platform.
5. Delivery is confirmed against the platform, and health figures are collected afterwards.

When something goes wrong

SITUATION	RESPONSE
Issue fails its quality checks	It is held and never offered for approval. Fix the cause and reassemble. Do not bypass the gate.
Nobody approved in time	Escalated to the named contact. Never auto-approved. Skipping the week is an acceptable outcome.
Delivery failed repeatedly	Stops and escalates rather than retrying indefinitely. Configuration errors escalate at once, since retrying cannot fix them.
Wrong content went out	Stop switches on. Cancel at the platform if still in flight — cancel, never resume. Then decide whether a correction is warranted.
Approval link opened by a scanner	Nothing happens; this is the designed behaviour. Confirm the issue's state is unchanged and continue.
Complaint threshold breached	Warning level: investigate acquisition and content. Enforced ceiling: stop sending until reputation recovers.

Backup and recovery

The operational record and the archived issues are backed up on a schedule, held somewhere other than the machine that produced them, and — critically — restored periodically as a rehearsal. A backup that has never been restored should not be counted as a backup. Sent issues are immutable, so a restored archive is identical to what recipients received.

C Glossary

TERM	MEANING
Opt-in	The subscriber explicitly asked to receive this email. The only basis on which we send.
Email platform	The service that holds the list and physically sends the mail.
Sender reputation	Mailbox providers' running judgement of whether our mail is wanted. Determines inbox versus spam. Slow to build, quick to lose.
Domain authentication	DNS records letting a receiving system verify mail genuinely came from our domain and was not altered.
One-click unsubscribe	Machine-readable headers letting a mailbox provider offer its own unsubscribe control. Required of bulk senders.
Complaint rate	Share of recipients marking the message as spam. The single most important deliverability signal.
Suppression	Addresses permanently excluded — unsubscribed, complained, or repeatedly undeliverable.
Sunset policy	Retiring subscribers who have not engaged for an extended period, to protect sender reputation.
Actively exploited vulnerability	A security flaw confirmed to be under real-world attack, not merely theoretical. The basis of our patch board.

TERM	MEANING
Vulnerability identifier	The standard public reference for a specific security flaw, letting readers look it up independently.
Dry run	The system performing every step except actually sending. Our default state.
Seed recipients	A small set of addresses we control, used to inspect a real received message before any live send.
System of record	The one place holding the authoritative answer to what happened. Here, the control plane.
Frozen issue	An issue fixed at approval so that what was approved is exactly what is sent and archived.
Clipping	Mailbox providers truncating an over-long message, hiding the footer — including the legally required parts — behind a link.

DOCUMENT STATUS

This is a draft for review. The strategy in Parts I–III is presented for approval; the decisions in section 14 need named owners before build work can reach a live send. Both should be settled before this plan is treated as agreed.